

Estrategia de Control de Versiones: Plataforma DataForge IA

1. Introducción

El presente documento define la **Estrategia de Control de Versiones** para el proyecto **DataForge IA**, una plataforma web de análisis predictivo para PYMES. El objetivo es establecer un marco de trabajo robusto y consistente para la gestión del código fuente, facilitando la colaboración del equipo de desarrollo, el seguimiento de cambios, la integración continua y el despliegue eficiente. La adopción de estas directrices es fundamental para mantener la calidad del software y asegurar un ciclo de vida de desarrollo ágil y predecible.

2. Uso de Git

Git será el sistema de control de versiones distribuido principal para todo el código fuente del proyecto DataForge IA. Se utilizará un repositorio centralizado (ej. GitHub, GitLab, Bitbucket) para alojar el código, permitiendo a los desarrolladores clonar, fusionar y contribuir de manera colaborativa. La naturaleza distribuida de Git ofrece flexibilidad, resiliencia y un historial completo de todos los cambios [1].

2.1. Configuración de Git

- **Identidad:** Todos los desarrolladores configurarán su nombre de usuario y correo electrónico de Git para asegurar la trazabilidad de los commits.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@ejemplo.com"
```

- **.gitignore** : Se mantendrá un archivo `.gitignore` actualizado en la raíz del repositorio para excluir archivos generados automáticamente, dependencias de paquetes, credenciales y otros archivos no esenciales para el control de versiones.

3. Flujo de Ramas (Git Flow)

Para gestionar el desarrollo de manera estructurada, se adoptará el modelo de ramificación **Git Flow** [2]. Este modelo define un conjunto de ramas con propósitos específicos y un proceso

claro para la gestión de nuevas funcionalidades, releases y hotfixes.

3.1. Ramas Principales

- **main (o master)**: Contiene el código de producción, siempre estable y listo para ser desplegado. Los commits a esta rama deben ser mínimos y solo provienen de fusiones de **release** o **hotfix**.
- **develop**: Contiene el código de desarrollo más reciente, que se integra de todas las ramas de **feature**. Es la rama base para el desarrollo de nuevas funcionalidades.

3.2. Ramas de Soporte

- **feature/<nombre-característica>**: Se crean a partir de **develop** para desarrollar nuevas funcionalidades. Una vez completadas, se fusionan de nuevo en **develop**.
 - **Convención de Nomenclatura**: **feature/HU-XXX-descripcion-corta** (donde HU-XXX es el ID de la Historia de Usuario).
- **release/<version>**: Se crean a partir de **develop** cuando se planea una nueva versión de producción. Se utilizan para realizar pruebas finales, correcciones de errores menores y preparación para el lanzamiento. Una vez estable, se fusiona en **main** y **develop**.
 - **Convención de Nomenclatura**: **release/v1.0.0**.
- **hotfix/<nombre-hotfix>**: Se crean a partir de **main** para corregir errores críticos en producción. Una vez corregido y probado, se fusiona en **main** y **develop**.
 - **Convención de Nomenclatura**: **hotfix/BUG-YYY-descripcion-corta**.

3.3. Proceso de Fusión (Merge) y Pull Requests (PRs)

- **Pull Requests (PRs)**: Todas las fusiones de ramas de **feature**, **release** y **hotfix** deben realizarse a través de Pull Requests. Esto garantiza la revisión de código por pares y la ejecución de pruebas automatizadas.
- **Revisión de Código**: Cada PR debe ser revisado y aprobado por al menos un miembro del equipo (idealmente dos) antes de la fusión.
- **Mensajes de Commit**: Se seguirán las convenciones de mensajes de commit definidas en los Estándares de Desarrollo, facilitando la trazabilidad y la generación de changelogs.

4. Control de Cambios

El control de cambios se implementará a través de las siguientes prácticas:

- **Trazabilidad:** Cada cambio en el código debe estar vinculado a una Historia de Usuario o un Bug en el sistema de gestión de proyectos (ej. Jira, Trello). Los mensajes de commit deben incluir la referencia a estos elementos.
- **Historial Inmutable:** Se evitará la reescritura del historial de Git en ramas compartidas (ej. `rebase` en `develop` o `main`). El historial debe ser lineal y reflejar la evolución real del código.
- **Etiquetado (Tagging):** Se utilizarán etiquetas (`tags`) en la rama `main` para marcar las versiones de producción (ej. `v1.0.0` , `v1.0.1`). Esto facilita la identificación de versiones específicas y la gestión de releases.

5. Integración Continua (CI)

La Integración Continua (CI) es un pilar fundamental para asegurar la calidad del código y la detección temprana de errores. Se implementará un pipeline de CI que se ejecutará automáticamente en cada push a las ramas de desarrollo y en cada Pull Request.

5.1. Herramientas de CI

Se utilizará **GitHub Actions** (o GitLab CI, Jenkins) para orquestar el pipeline de CI/CD.

5.2. Pasos del Pipeline de CI

El pipeline de CI incluirá, como mínimo, los siguientes pasos:

1. **Checkout del Código:** Obtener la última versión del código del repositorio.
2. **Instalación de Dependencias:** Instalar todas las librerías y paquetes necesarios para el proyecto.
3. **Análisis Estático de Código (Linting):** Ejecutar herramientas como `ESLint` (JavaScript/TypeScript) y `Flake8` (Python) para verificar el cumplimiento de las convenciones de código y detectar posibles errores.
4. **Ejecución de Pruebas Unitarias:** Ejecutar todas las pruebas unitarias del proyecto (`Pytest` para Python, `Jest` para JavaScript/TypeScript). Un fallo en las pruebas detendrá el pipeline.
5. **Ejecución de Pruebas de Integración:** Ejecutar pruebas que verifiquen la interacción entre los diferentes componentes del sistema.
6. **Construcción de Artefactos:** Compilar el código frontend (si aplica) y construir las imágenes Docker para los microservicios.
7. **Análisis de Seguridad:** Ejecutar herramientas de análisis de seguridad estático (SAST) para identificar vulnerabilidades en el código.

5.3. Notificaciones

El equipo recibirá notificaciones automáticas (ej. Slack, email) sobre el estado de las ejecuciones del pipeline de CI, alertando sobre fallos para una resolución rápida.

6. Conclusiones

La implementación de esta Estrategia de Control de Versiones, basada en Git Flow y complementada con un robusto pipeline de Integración Continua, es esencial para el éxito del proyecto DataForge IA. Permitirá al equipo de desarrollo trabajar de manera eficiente, colaborativa y con la confianza de que los cambios se integran y prueban de forma continua, resultando en un software de alta calidad, estable y listo para el despliegue. Esta estrategia fomenta la agilidad, la transparencia y la responsabilidad compartida en la gestión del código fuente.

7. Referencias

[1] Chacon, S., & Straub, B. (2014). *Pro Git*. Apress. [2] Vincent Driessen. (2010). *A successful Git branching model*. nvie.com.

- Documentación Técnica: DataForge IA.
- Estándares de Desarrollo: DataForge IA.